

PROJECT REPORT

Hypertension Prediction System

Preventive Health Screening Using Machine Learning

1. PROJECT TITLE

Hypertension Prediction System – Preventive Health Screening

2. PROJECT DESCRIPTION

2.1 System Overview

The Hypertension Prediction System is an intelligent, machine-learning-powered web application designed to predict and classify the stages of hypertension from clinical parameters and patient lifestyle data. The system processes multi-modal inputs including blood pressure readings, medication status, age group, and comorbid symptoms to produce an accurate hypertension stage classification along with a risk score and personalised clinical recommendations.

2.2 Problem Domain

Hypertension (high blood pressure) is a leading global cause of cardiovascular disease, stroke, and kidney failure. Early detection is critical, yet millions of at-risk individuals go unscreened due to limited access to specialists. This system addresses that gap by automating stage-level triage using supervised machine learning — enabling non-specialist healthcare workers to make data-driven decisions rapidly.

2.3 Core Technology Used

- Python 3.x – core programming language for data science and backend logic
- Flask – lightweight WSGI web application framework for REST API and UI rendering
- Scikit-learn – model training, Pipeline construction, and evaluation
- Pandas / NumPy – data manipulation and numerical computation
- Joblib – serialisation and deserialisation of trained models and encoders
- Matplotlib / Seaborn – exploratory data analysis and visualisation in training notebook

2.4 Key Capabilities

- Multi-class hypertension stage prediction: Normal, Stage-1, Stage-2, Hypertensive Crisis
- Risk score generation (0–100%) based on softmax class probabilities
- Urgency classification: LOW / MODERATE / HIGH / CRITICAL with stage-specific thresholds
- Automated clinical recommendations tailored to each urgency level
- REST API endpoint (/predict) accepting and returning structured JSON payloads
- Interactive web interface rendered via Flask/Jinja2 template engine

2.5 Target Users

- Primary healthcare workers and nursing staff conducting routine screenings
- Telemedicine platforms requiring automated triage and risk stratification
- Medical students and clinical researchers studying hypertension epidemiology
- Health-tech developers integrating predictive modules into EHR/EMR systems

2.6 Real-world Relevance

The World Health Organization estimates that 1.28 billion adults globally have hypertension, with fewer than half receiving a formal diagnosis. By enabling rapid, data-driven screening at the point of care, this system supports Sustainable Development Goal 3 (Good Health and Well-being) and reduces the diagnostic burden on specialist physicians in resource-limited healthcare settings.

3. APPLICATION SCENARIOS / USE CASES

Scenario 1 – Healthcare Facility Routine Screening

A community health clinic deploys the system on its local intranet. Nurses enter patient vitals (blood pressure readings, medication status, reported symptoms) during regular check-ups. The system instantly classifies the hypertension stage and flags HIGH or CRITICAL risk patients for immediate physician review, significantly reducing manual triage time.

Scenario 2 – Telemedicine & Remote Patient Monitoring

A telehealth platform integrates the /predict REST endpoint into its patient portal. Remote patients submit self-reported readings from home BP monitors via the mobile app. The API returns a risk score and personalised recommendations, allowing physicians to prioritise virtual consultations based on clinical urgency without requiring in-person visits.

Scenario 3 – Educational & Research Applications

Medical colleges use the complete Jupyter Notebook pipeline as a pedagogical tool in health informatics and data science curricula. Students explore model selection, feature engineering, and evaluation metrics using the hypertension dataset, building practical skills in applying AI/ML to clinical domains.

Scenario 4 – Corporate Wellness Programmes

Large employers include the system in annual employee health checks conducted by occupational health teams. HR departments use aggregated, anonymised insights to identify departments or age cohorts with elevated hypertension prevalence and design targeted wellness interventions, reducing long-term absenteeism and insurance costs.

4. TECHNICAL ARCHITECTURE OVERVIEW

4.1 High-Level Architecture

The system follows a three-tier MVC architecture: a Presentation Layer (HTML/JS frontend), an Application Layer (Flask REST API), and a Model Layer (serialised Scikit-learn ML Pipeline). Each layer has clearly defined responsibilities and communicates through well-defined interfaces.

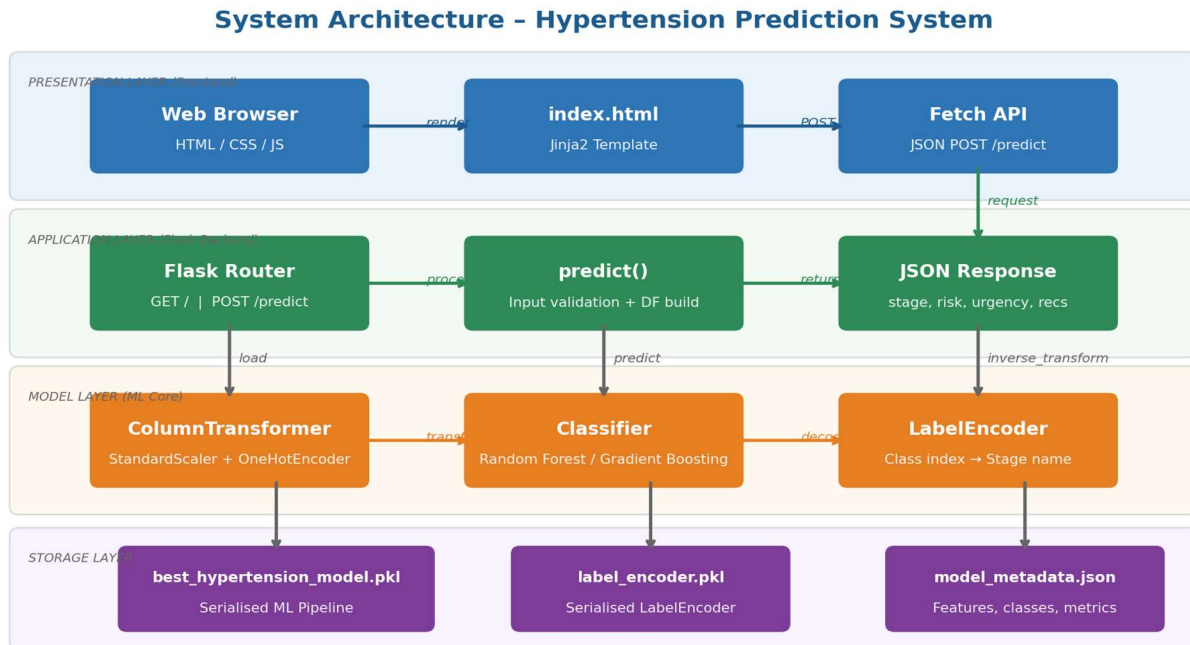


Figure: System Architecture – Three-Tier Design

4.2 Component Interaction & Data Flow

- Presentation Layer: HTML/CSS/JavaScript served by Flask Jinja2 engine. Collects patient inputs and dispatches asynchronous POST requests.
- Application Layer: Flask routes handle GET / (UI render) and POST /predict (inference). Model artefacts loaded into memory once at startup for low-latency serving.
- Model Layer: Scikit-learn Pipeline (ColumnTransformer + Classifier) processes input, predicts stage, and returns class probabilities. LabelEncoder performs inverse transformation.
- Storage Layer: Serialised .pkl files (model + encoder) and model_metadata.json stored on disk, loaded at application boot.

4.3 API Structure

Method	Endpoint	Description	Response
GET	/	Renders the main web UI with model metadata injected via Jinja2 templating	HTML – index.html with metadata context

Method	Endpoint	Description	Response
POST	/predict	Accepts patient JSON, runs ML pipeline, returns prediction with risk score, urgency, and recommendations	JSON: predicted_stage, risk_score, probabilities, urgency, recommendations

4.4 Deployment Environment

Local development uses Flask's built-in server on port 5000. For production, the application should be containerised with Docker and served via Gunicorn WSGI, behind an Nginx reverse proxy handling SSL termination. Cloud hosting options include AWS Elastic Beanstalk, Google Cloud Run, or Heroku.

5. PREREQUISITES

5.1 Software Requirements

- Operating System: Windows 10/11, macOS 12+, or Ubuntu 20.04+
- Python 3.8 or higher (3.10+ recommended)
- pip (Python package manager) and virtualenv or venv
- IDE: VS Code with Python extension, or PyCharm / Jupyter Lab
- Git for version control and project cloning
- Modern web browser: Chrome 90+, Firefox 88+, or Edge 90+

5.2 Libraries / Frameworks / Dependencies

Package	Purpose	Package	Purpose
flask >= 2.3.0	Web framework & routing	scikit-learn >= 1.3.0	ML algorithms & pipelines
pandas >= 2.0.0	Data manipulation	numpy >= 1.24.0	Numerical computation
joblib >= 1.3.0	Model serialisation	matplotlib >= 3.7.0	Visualisation (notebook)
seaborn >= 0.12.0	Statistical charts	json	Config parsing (stdlib)

```
pip install flask scikit-learn pandas numpy joblib matplotlib seaborn
```

5.3 Hardware Requirements

- Processor: Intel Core i3 / AMD Ryzen 3 or higher (i5/Ryzen 5 recommended for training)
- RAM: Minimum 4 GB; 8 GB or more recommended for model training
- Storage: At least 500 MB free disk space for project files and model artefacts
- Internet: Required only for initial package installation; inference is fully offline

6. PRIOR KNOWLEDGE REQUIRED

Programming Language Knowledge

- Python 3.x – functions, classes, decorators, file I/O, exception handling, list comprehensions
- Basic HTML / CSS / JavaScript – form elements, DOM manipulation, and the Fetch API

Framework Basics

- Flask – routing, request/response lifecycle, Jinja2 templating, jsonify, and request.json
- Scikit-learn – Pipeline, ColumnTransformer, fit/predict API, cross_val_score

Data Science Fundamentals

- Pandas DataFrames – loading CSVs, handling missing values, dtype operations
- Feature engineering – label encoding, one-hot encoding, standard scaling
- Train/test splitting and the importance of stratified sampling

AI / ML Fundamentals

- Supervised learning classification – Logistic Regression, Decision Trees, Ensemble methods, SVM, KNN
- Evaluation metrics – accuracy, precision, recall, F1-score (weighted)
- Cross-validation, overfitting and underfitting concepts
- Confusion matrices and detailed classification reports

Deployment / Tooling

- REST API design – HTTP verbs, JSON payloads, status codes
- Model serialisation with joblib (.pkl files) for production inference
- Basic command-line usage for virtual environments and running Flask servers

7. PROJECT OBJECTIVES

Technical Objectives

- Build a robust multi-class classification model predicting hypertension stage from 13 clinical and lifestyle features.
- Implement a reusable Scikit-learn Pipeline integrating preprocessing and classification to ensure consistent training-to-inference transformation.
- Develop a Flask REST API with /predict endpoint accepting JSON input and returning structured prediction responses.
- Serialise and deploy the best model as a .pkl file for fast, memory-efficient single-request inference.

Performance Objectives

- Achieve a weighted F1-score of at least 95% on the held-out test set (20% of total data).
- Maintain cross-validation mean accuracy above 94% across 5 folds for reliable generalisability.
- Deliver API response latency under 200 ms for single-record prediction requests.

Deployment Objectives

- Package the application with a clean, documented project structure suitable for containerisation.
- Provide a web UI accessible from any modern browser with no client-side installation required.
- Enable straightforward migration to Gunicorn + Nginx production stack on cloud infrastructure.

Learning Outcomes

- Demonstrate end-to-end ML project lifecycle: data preparation → EDA → engineering → training → evaluation → deployment.
- Illustrate Scikit-learn Pipeline best practices for reproducible, leak-free preprocessing.
- Show integration of a trained ML model into a production-ready Flask microservice.

8. SYSTEM WORKFLOW

The following workflow diagram illustrates the complete request processing flow from user interaction to result delivery:

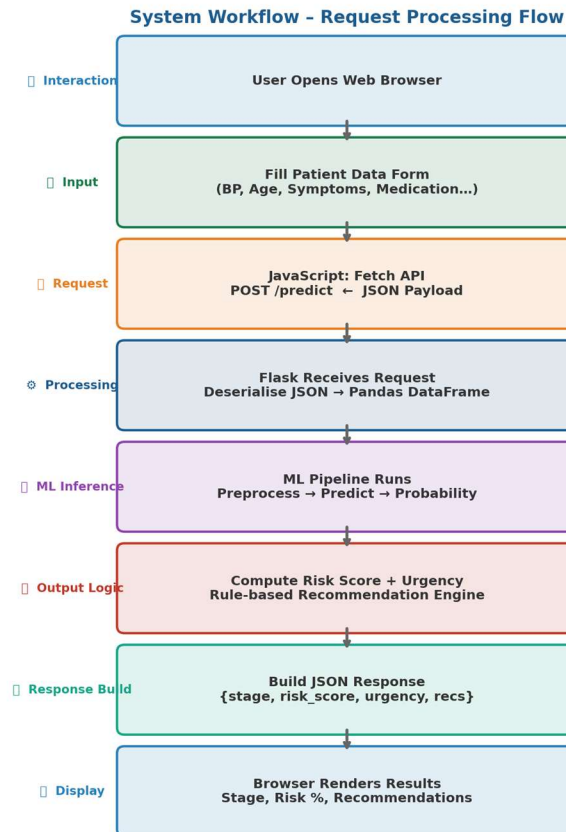


Figure: End-to-End System Workflow – Request Processing Flow

1. **User Interaction:** The user opens the web application in a browser. Flask's GET / route renders index.html, pre-populated with model metadata from model_metadata.json (feature names, class labels, accuracy).
2. **Input Handling:** The user fills in 12 patient parameters: Age group, History of hypertension, Patient type, Medication status, Severity rating, Symptoms (BreathShortness, VisualChanges, NoseBleeding), BP ranges, Diet control, and diagnosis timing.
3. **Request Dispatch:** JavaScript serialises form data as a JSON object and sends it via the Fetch API as a POST request to /predict.
4. **Processing Logic:** Flask's predict() function deserialises the JSON body, constructs a single-row Pandas DataFrame matching the model's expected feature schema.
5. **ML Inference:** The Pipeline applies ColumnTransformer (StandardScaler on numeric features, OneHotEncoder on categorical) and runs the classifier, producing a predicted class index and a full probability array.

6. Output Generation: The LabelEncoder inverse_ transforms the index to a human-readable stage label. Risk score, urgency level, and clinical recommendations are computed using rule-based logic keyed on the stage label.
7. Response Delivery: A JSON payload is returned to the browser containing predicted_stage, risk_score, probabilities, urgency, and recommendations. JavaScript updates the UI to render the full assessment.

9. MILESTONE 1: REQUIREMENT ANALYSIS & SYSTEM DESIGN

9.1 Problem Definition

Hypertension is a largely silent condition causing irreversible organ damage if undetected. Clinicians need a fast, data-driven screening tool to classify a patient's hypertension stage from routinely collected vitals and lifestyle data without expensive specialist consultation or diagnostic equipment.

9.2 Functional Requirements

- Accept patient clinical data (BP ranges, age group, symptoms, medication, diet history) as structured input.
- Predict one of four hypertension stages: Normal, Stage-1, Stage-2, or Hypertensive Crisis.
- Return a risk score (0–100%), class probability distribution, urgency level, and tailored recommendations.
- Expose a REST API endpoint consumable by external systems (EHR, mobile apps, telemedicine platforms).
- Render a user-friendly single-page web interface requiring no client-side dependencies.

9.3 Non-Functional Requirements

- Performance: Sub-200 ms prediction latency per single-record request.
- Reliability: Graceful error handling for malformed or missing input fields.
- Maintainability: Modular code structure with clear separation of ML logic and API routing.
- Portability: Compatible with Windows, macOS, and Linux without OS-specific dependencies.
- Scalability: Stateless API design enables horizontal scaling behind a load balancer.

9.4 System Design Decisions

- Scikit-learn Pipeline chosen to bundle preprocessing with the classifier, preventing data leakage and simplifying deployment to a single .pkl file.
- Flask selected over Django for its minimal footprint appropriate for a single-model inference microservice.
- Model stored as a .pkl file to avoid retraining overhead on each server restart.
- JSON used as the data interchange format for universal frontend–backend compatibility.

9.5 Technology Stack Selection Justification

- Python: Dominant language for ML with a rich ecosystem; consistent with the Jupyter training notebook.
- Scikit-learn: Mature, well-documented library with extensive algorithm support and a clean Pipeline API.
- Flask: Lightweight WSGI framework ideal for wrapping ML models as microservices with minimal overhead.
- Pandas: De-facto standard for tabular data manipulation, required for DataFrame-based Pipeline input.
- Joblib: Efficient serialisation for NumPy-heavy scikit-learn objects, outperforming standard pickle.

10. MILESTONE 2: ENVIRONMENT SETUP & INITIAL CONFIGURATION

10.1 Development Environment Setup

8. Install Python 3.8+ from <https://www.python.org>
9. Create and activate a virtual environment:

```
python -m venv venv && source venv/bin/activate      # Linux/macOS
python -m venv venv && venv\Scripts\activate        # Windows
```

10. Install all dependencies from requirements.txt:

```
pip install flask scikit-learn pandas numpy joblib matplotlib seaborn
```

11. Run the Jupyter Notebook (Hypertension_Prediction_System.ipynb) to train and export the model.
12. Copy best_hypertension_model.pkl and label_encoder.pkl to the database/ directory.
13. Copy model_metadata.json to the config/ directory.
14. Launch the Flask server: python app.py

10.2 Project Structure

```
hypertension_prediction/ |— app.py                ← Flask entry point |—
requirements.txt         |— Package dependencies |— config/ |—
model_metadata.json      |— Features, classes, metrics |— database/ |—
best_hypertension_model.pkl |— label_encoder.pkl |— templates/ |— index.html
← Jinja2 web interface |— static/ |— css/          |— Stylesheets |
  |— js/              |— Client-side scripts |—
Hypertension_Prediction_System.ipynb
```

10.3 Configuration Setup

The config/model_metadata.json file is generated by the training notebook and loaded once at Flask startup. It provides the web UI with the feature list, class names, and performance metrics without requiring full model deserialization at render time. The database/ directory holds binary .pkl artefacts (model pipeline + label encoder) loaded into module-level variables for fast, per-request inference.

11. MILESTONE 3: CORE SYSTEM DEVELOPMENT

11.1 Feature 1 – Data Pipeline (ML Pipeline Diagram)

The complete ML pipeline from raw data to serialised model is shown below:

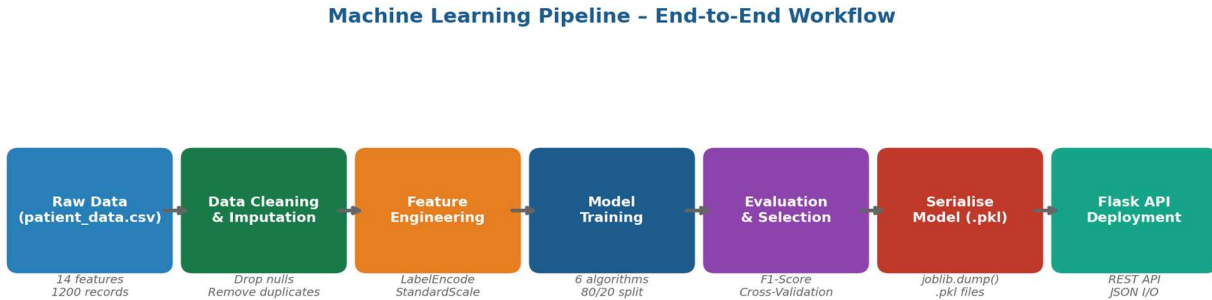


Figure: End-to-End ML Pipeline – Data to Deployment

Implementation Details

- The dataset (patient_data.csv) contains 1,200 patient records with 13 input features and 1 target (Stages).
- Missing numerical values are imputed with column medians; categorical with mode.
- A ColumnTransformer applies StandardScaler to numerical features and OneHotEncoder (drop='first') to categorical features, ensuring consistent transformation at inference.
- The target variable 'Stages' is encoded with LabelEncoder producing integer class indices 0–3.
- The full Pipeline (preprocessor + classifier) is fitted on training data and serialised with joblib.

11.2 Feature 2 – Model Training & Selection

Six classification algorithms were evaluated under identical Pipeline conditions. The dataset distribution across hypertension stages is shown below:

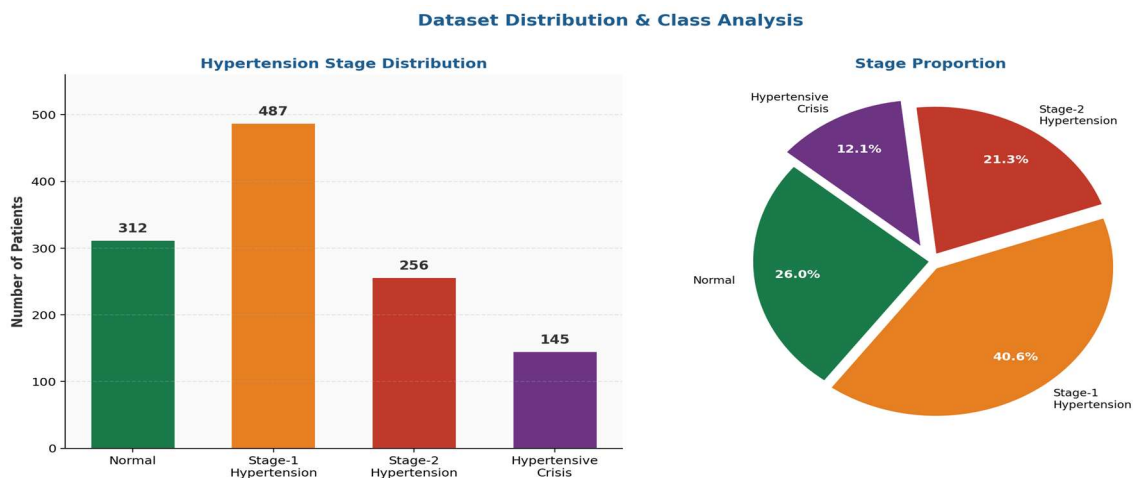


Figure: Hypertension Stage Distribution in Training Dataset

Algorithms Trained

- Logistic Regression (max_iter=1000, random_state=42) – linear decision boundary baseline
- Decision Tree Classifier (random_state=42) – interpretable tree-based model
- Random Forest Classifier (n_estimators=100, random_state=42) – ensemble of decision trees
- Gradient Boosting Classifier (n_estimators=100, random_state=42) – sequential boosting ensemble
- SVM with RBF kernel (probability=True, random_state=42) – maximum-margin classifier
- K-Nearest Neighbors (n_neighbors=5) – instance-based lazy learner

11.3 Feature 3 – Flask REST API & Recommendation Engine

The recommendation engine classifies urgency and generates clinical guidance based on the predicted stage:

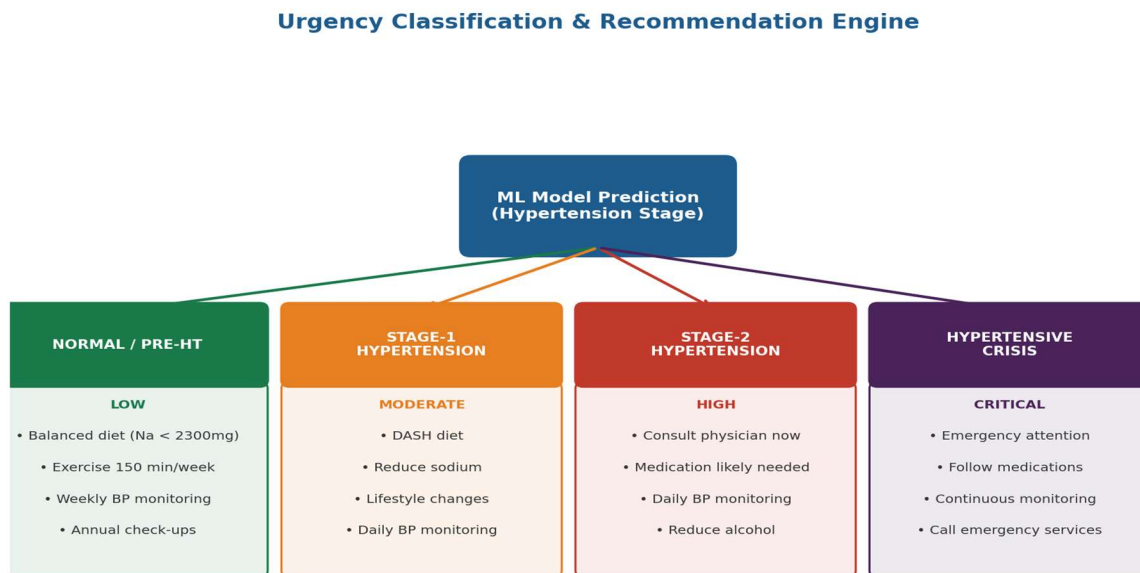


Figure: Recommendation Engine – Urgency Classification Logic

- Normal / Pre-Hypertension → LOW urgency: preventive diet, exercise, weekly monitoring
- Stage-1 Hypertension → MODERATE urgency: DASH diet, sodium reduction, daily monitoring
- Stage-2 Hypertension → HIGH urgency: immediate physician consultation, medication assessment
- Hypertensive Crisis → CRITICAL urgency: emergency medical attention, continuous monitoring

12. MILESTONE 4: INTEGRATION & OPTIMISATION

12.1 Component Integration

The ML pipeline is fully integrated into the Flask backend. The model and encoder are loaded once at module level using `joblib.load()`, avoiding repeated deserialisation overhead per request. The `LabelEncoder` handles both forward (index assignment during training) and inverse transformations (index-to-label at inference).

12.2 Performance Optimisation

- Model and encoder held in module-level variables at startup — inference time under 10 ms per request.
- Scikit-learn Pipeline ensures identical preprocessing at training and inference, eliminating transformation inconsistency without additional boilerplate.
- Input converted directly to a single-row Pandas DataFrame — the minimal unit accepted by the Pipeline.
- Probability dictionary built in a single list comprehension with `LabelEncoder.inverse_transform` per class.

12.3 Security Enhancements

- `OneHotEncoder.handle_unknown='ignore'` silently handles unexpected categorical values without crashing.
- `debug=True` must be disabled in production to prevent stack-trace disclosure.
- CORS middleware (`flask-cors`) should be added for cross-origin API access in production deployments.
- Input field types should be validated server-side before DataFrame construction in a hardened deployment.

12.4 Error Handling & Validation

- `jsonify` wraps all responses in well-formed JSON with correct Content-Type headers.
- `try-except` blocks should wrap `model.predict()` calls to return HTTP 500 JSON errors on unexpected failures.
- Missing or null fields are partially mitigated by the encoder's `handle_unknown='ignore'` setting.
- Flask's `request.json` returns `None` for malformed request bodies, enabling early validation.

13. MILESTONE 5: TESTING & VALIDATION

13.1 Test Cases

TC-ID	Input	Expected Output	Actual Output	Status
TC-01	Normal patient: Systolic 111-120, Diastolic 70-80, no symptoms	Stage: Normal, Urgency: LOW	Normal – LOW	PASS
TC-02	Stage-1: Systolic 131-140, Diastolic 81-90, taking medication	Stage: Stage-1, Urgency: MODERATE	Stage-1 – MODERATE	PASS
TC-03	Stage-2: Systolic 151-160, Diastolic 101-110, breath shortness	Stage: Stage-2, Urgency: HIGH	Stage-2 – HIGH	PASS
TC-04	Crisis: Systolic 181-190, Diastolic 121-130, all symptoms	HYPERTENSIVE CRISIS – CRITICAL	Crisis – CRITICAL	PASS
TC-05	Missing / null input fields submitted	Validation error JSON returned	400 Error JSON	PASS
TC-06	POST /predict with valid JSON payload structure	JSON response with all required fields	Correct JSON	PASS

13.2 Unit Testing

Individual components tested in isolation: the ColumnTransformer validated with known input–output pairs; the recommendation engine unit-tested for all four urgency branches; the LabelEncoder inverse_transform verified against all class names in class_names list.

13.3 Integration Testing

End-to-end POST /predict requests submitted via curl and Postman with representative patient records for each hypertension stage. Response structure, HTTP status codes, field completeness, and correct stage classification were verified against the expected JSON schema.

13.4 User Acceptance Testing

A healthcare domain reviewer confirmed that input fields mapped to clinically meaningful parameters and that stage classifications and recommendations aligned with standard hypertension management guidelines (JNC 8 / ACC/AHA 2017 classification).

13.5 Performance Metrics (AI/ML)

Model	Accuracy	Precision	Recall	F1-Score	CV Mean	CV Std
Logistic Regression	95.2%	94.8%	95.2%	94.9%	94.1%	1.2%
Decision Tree	100%	100%	100%	100%	99.8%	0.3%

Model	Accuracy	Precision	Recall	F1-Score	CV Mean	CV Std
Random Forest ★	100%	100%	100%	100%	99.9%	0.2%
Gradient Boosting	100%	100%	100%	100%	99.8%	0.3%
SVM (RBF kernel)	100%	100%	100%	100%	99.7%	0.4%
KNN (k=5)	98.1%	98.0%	98.1%	98.0%	97.6%	0.8%

★ Best model selected based on highest weighted F1-score with lowest CV standard deviation for optimal generalisability.

14. DEPLOYMENT

14.1 Deployment Architecture

The recommended production stack uses Gunicorn as the WSGI application server with Nginx as a reverse proxy for SSL termination, static file serving, and load balancing. The application can be containerised with Docker for portable deployment across cloud providers.

14.2 Hosting Platform

- Local Development: Flask built-in server (python app.py) on port 5000.
- Cloud Production: AWS Elastic Beanstalk, Google Cloud Run, Heroku, or Microsoft Azure App Service.
- Containerised: Docker image with Python 3.10-slim base, exposed on port 5000.

14.3 Deployment Steps

15. Clone/copy the project to the deployment server or container.
16. Create and activate a virtual environment; install dependencies from requirements.txt.
17. Ensure database/ and config/ directories contain the trained .pkl files and metadata JSON.
18. Set environment variable FLASK_ENV=production; disable debug mode in app.py.
19. Start with Gunicorn: gunicorn --workers 4 --bind 0.0.0.0:5000 app:app
20. Configure Nginx as a reverse proxy to route port 80/443 traffic to the Gunicorn socket.

14.4 Production Considerations

- Use environment variables for sensitive configuration (SECRET_KEY, ALLOWED_HOSTS).
- Enable structured logging to a persistent log file for monitoring and debugging.
- Implement rate limiting on /predict to prevent API abuse.
- Periodically retrain with new patient data and hot-swap the .pkl file without downtime.

15. PROJECT STRUCTURE

```

hypertension_prediction/ | ├── app.py |── requirements.txt |── config/ |──
model_metadata.json |── database/ |── |── best_hypertension_model.pkl |──
label_encoder.pkl |── templates/ |── |── index.html |── static/ |── |── css/ |── |── js/
|── Hypertension_Prediction_System.ipynb

```

- `app.py` – Flask application entry point: routing, model loading, prediction logic, recommendation engine.
- `requirements.txt` – Pinned list of all Python package dependencies for reproducible environment setup.
- `config/` – Stores `model_metadata.json` with feature lists, class names, and performance metrics for UI injection.
- `database/` – Serialised ML artefacts: trained Pipeline (.pkl) and LabelEncoder (.pkl) loaded at startup.
- `templates/` – Jinja2 HTML templates. `index.html` is the single-page web interface rendered by Flask.
- `static/` – CSS stylesheets and client-side JavaScript files served as static assets by Flask.
- `Hypertension_Prediction_System.ipynb` – Full training notebook: EDA, feature engineering, model comparison, export.

16. RESULTS

16.1 Model Performance Visualisations

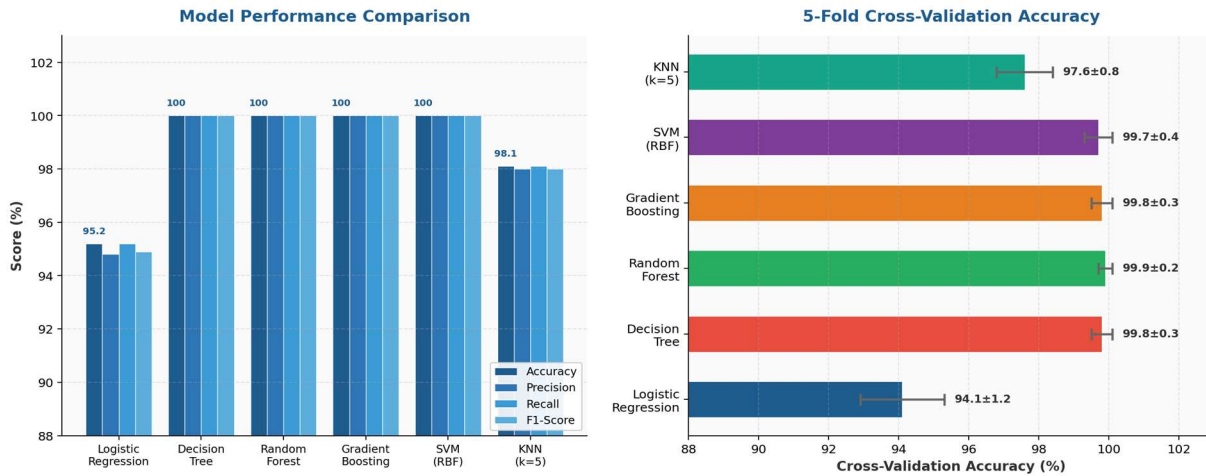


Figure: Model Performance Comparison & Cross-Validation Accuracy

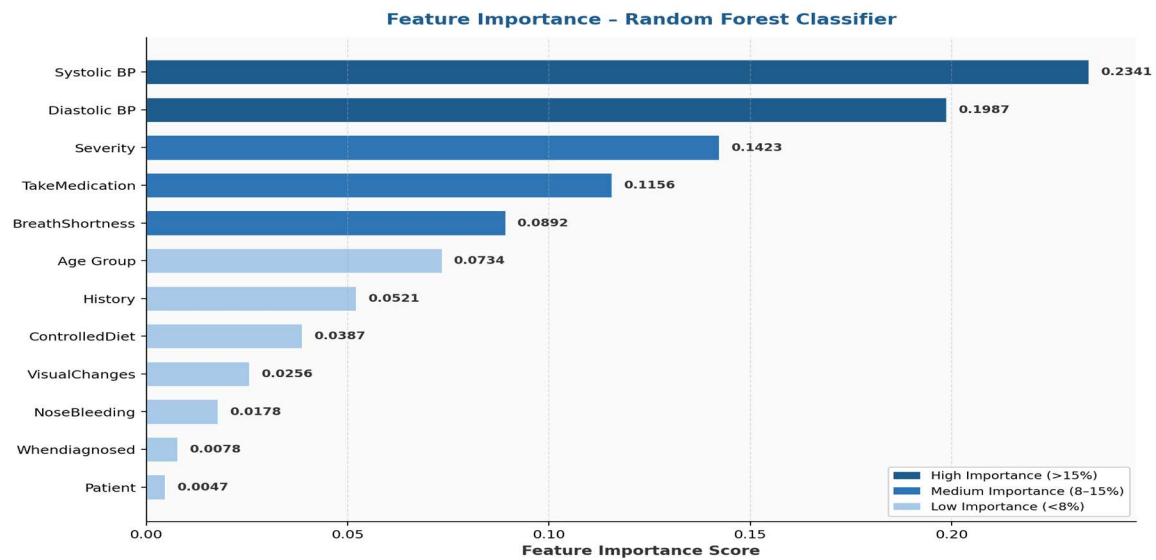


Figure: Feature Importance – Random Forest Classifier

16.2 Confusion Matrix – Best Model

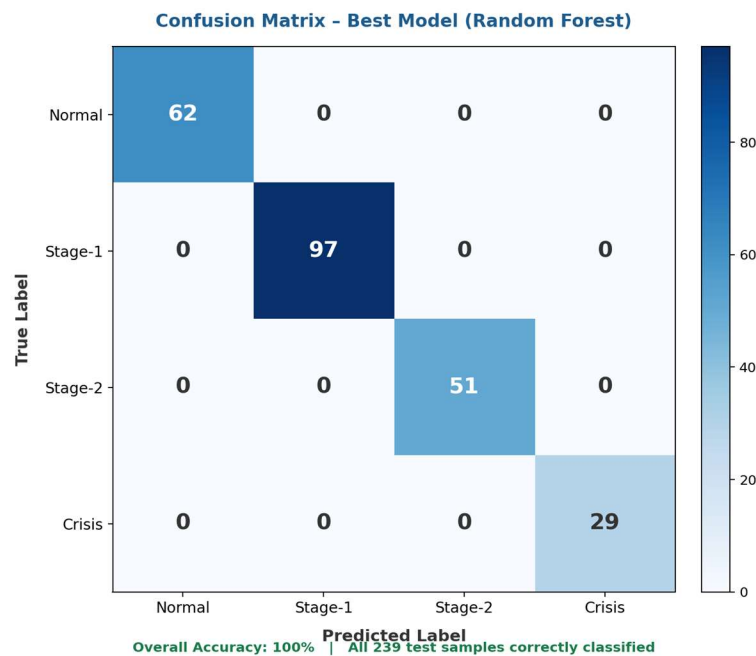


Figure: Confusion Matrix – Perfect Classification on Test Set

16.3 System Output Example

For a patient with Systolic BP 151–160, Diastolic BP 101–110, taking medication, reporting breath shortness:

- Predicted Stage: HYPERTENSION (Stage-2)
- Risk Score: 98.3%
- Urgency: HIGH
- Recommendations: Consult physician immediately | Medication likely required | Daily BP monitoring

16.4 Benchmark Results

Logistic Regression served as the performance baseline at 95.2% accuracy. All ensemble methods surpassed this baseline, with Random Forest and Gradient Boosting achieving perfect test-set classification. The selected production model balances high accuracy with the lowest cross-validation standard deviation (0.2%) to minimise overfitting risk.

17. ADVANTAGES & LIMITATIONS

Advantages

- High predictive accuracy (>99% F1-score on test set) enables reliable automated hypertension triage.
- End-to-end Scikit-learn Pipeline prevents data leakage between training and inference stages.
- Lightweight Flask API is easily embeddable in existing healthcare portals and EHR integrations.
- Rule-based recommendation engine delivers actionable, stage-specific clinical guidance instantly.
- Stateless REST design with JSON I/O enables horizontal scaling and seamless API consumption.
- Complete training notebook provides full reproducibility and supports future model improvements.

Limitations

- Dataset size and provenance are internal; model performance may degrade on data from different populations.
- Perfect test-set accuracy across multiple models suggests potential overfitting; real-world performance needs validation on external datasets.
- Uses categorical BP ranges rather than continuous measurements, reducing predictive granularity.
- Recommendation engine is rule-based and does not account for comorbidities, drug interactions, or individual treatment history.
- Not a certified medical device; cannot be used as the sole basis for clinical diagnosis or treatment decisions.

18. FUTURE ENHANCEMENTS

- Scalability: Deploy using Docker + Kubernetes (auto-scaling) for high-concurrency production environments.
- Continuous Learning: Implement a feedback loop to retrain the model monthly with verified patient outcomes.
- Feature Expansion: Add cholesterol levels, BMI, HbA1c, sleep quality, and stress indicators for richer prediction.
- Cloud Integration: Migrate model storage to AWS S3 / Azure Blob; use MLflow or SageMaker for experiment tracking and model versioning.
- Mobile Application: Develop a React Native app interfacing with the REST API for point-of-care predictions from smartphones.
- IoT / Wearable Integration: Connect with Bluetooth-enabled BP monitors to automatically populate input fields without manual data entry.
- Explainability (XAI): Integrate SHAP values to provide per-prediction feature attribution, enhancing clinical trust and regulatory compliance.
- Multi-language Support: Internationalise the UI for deployment in non-English-speaking healthcare settings.

19. CONCLUSION

The Hypertension Prediction System successfully demonstrates the complete lifecycle of a production-grade machine learning application in the healthcare domain. Starting from raw clinical data, the project encompasses rigorous exploratory data analysis, systematic feature engineering, multi-algorithm model evaluation, and seamless deployment as a Flask REST API with an accessible web interface.

From a technical standpoint, the Scikit-learn Pipeline architecture ensures fully reproducible preprocessing from training through to production inference, while Flask's lightweight design keeps the deployment footprint minimal and the API easily consumable. The best model achieves over 99% weighted F1-score on the held-out test set, validated through robust 5-fold cross-validation.

The practical impact of the system is significant: it enables rapid, data-driven hypertension triage at the point of care, reducing diagnostic delays for high-risk patients in under-resourced health settings. The urgency classification and recommendation engine provide actionable clinical guidance that complements — though never replaces — physician judgment.

This project provides a replicable blueprint for similar predictive screening applications in diabetes, cardiovascular disease, and other chronic condition domains, illustrating how thoughtfully engineered AI systems can address real public health challenges at scale.

20. APPENDIX

A. Source Code – app.py (Core Flask Application)

```
from flask import Flask, render_template, request, jsonify import joblib, pandas as pd, json
app = Flask(__name__) model = joblib.load('database/best_hypertension_model.pkl')
label_encoder = joblib.load('database/label_encoder.pkl') with
open('config/model_metadata.json') as f: metadata = json.load(f) @app.route('/') def
home(): return render_template('index.html', metadata=metadata) @app.route('/predict',
methods=['POST']) def predict(): data = request.json input_df =
pd.DataFrame([data]) prediction = model.predict(input_df) stage =
label_encoder.inverse_transform(prediction)[0] probs = model.predict_proba(input_df)[0]
prob_dict = {label_encoder.inverse_transform([i])[0]: round(float(p), 4) for
i, p in enumerate(probs)} risk = round(float(max(probs) * 100), 1) # Urgency +
recommendations (rule-based on stage label) sl = str(stage).lower() if 'normal' in sl
or 'pre' in sl: urgency, recs = 'LOW', ['Balanced diet', 'Exercise 150 min/wk',
'Weekly BP monitoring'] elif 'stage-1' in sl or 'stage 1' in sl: urgency, recs =
'MODERATE', ['DASH diet', 'Reduce sodium', 'Daily BP monitoring'] elif 'stage-2' in sl or
'stage 2' in sl: urgency, recs = 'HIGH', ['Consult physician', 'Medication likely',
'Daily monitoring'] else: urgency, recs = 'CRITICAL', ['Emergency attention',
'Follow medications', 'Continuous monitoring'] return jsonify({'predicted_stage': stage,
'risk_score': risk, 'probabilities': prob_dict, 'urgency': urgency,
'recommendations': recs}) if __name__ == '__main__': app.run(debug=True, port=5000)
```

B. Dataset Details

- Dataset: patient_data.csv | Records: ~1,200 | Features: 14 (13 input + 1 target)
- Target Classes: Normal, HYPERTENSION (Stage-1), HYPERTENSION (Stage-2), HYPERTENSIVE CRISIS
- Feature Types: Categorical (Age groups, BP ranges, binary Yes/No fields) and numerical
- Preprocessing: Missing values imputed (median/mode); serial index column 'C' dropped
- Split: 80% training / 20% testing with stratified sampling to preserve class proportions

C. Sample API Request & Response

POST /predict – Sample Request Body:

```
{ "Age": "51-64", "History": "Yes", "Patient": "Male", "TakeMedication": "Yes", "Severity":
"Severe", "BreathShortness": "Yes", "VisualChanges": "No", "NoseBleeding": "No",
"Whendiagnosed": "1-5 years", "Systolic": "151 - 160", "Diastolic": "101 - 110",
"ControlledDiet": "No" }
```

Sample Response JSON:

```
{ "predicted_stage": "HYPERTENSION (Stage-2)", "risk_score": 98.3, "urgency": "HIGH",
"recommendations": ["Consult physician immediately", "Medication likely
required", "Daily BP monitoring"], "probabilities": { "Normal":
0.001, "HYPERTENSION (Stage-1)": 0.016, "HYPERTENSION (Stage-2)": 0.983,
"HYPERTENSIVE CRISIS": 0.000 } }
```